

MEDI TWIN



**THESIS SUBMITTED TOWARDS THE PARTIAL FULFILMENT OF THE REQUIRE-
MENT OF THE UNIVERSITY OF SINDH, FOR THE AWARD OF BACHELOR OF SCI-
ENCE IN INFORMATION TECHNOLOGY**

SUBMITTED BY

NAME: MUHAMMAD SUBHAN KHAN

SEAT NUMBER: 2k22/IT/81

NAME: SYED KHUSHAM SHAH

SEAT NUMBER: 2k22/IT/121

NAME: ALISHBA MEMON

SEAT NUMBER: 2k22/IT/18

SUPERVISOR

Dr. MUHAMMAD ALI MEMON
Associate Professor IICT

CO-SUPERVISOR

Dr. MUHAMMAD ALI NIZAMANI
Associate Professor IICT

**DEPARTMENT OF INFORMATION TECHNOLOGY
FACULTY OF ENGINEERING AND TECHNOLOGY**

UNIVERSITY OF SINDH

Session 2025

CERTIFICATE



TITLE OF PROJECT

A Thesis Submitted By

NAME: M. SUUBHAN KHAN

SEAT NUMBER 2k22/IT/81

NAME: SYED KHSUAHM SHAH

SEAT NUMBER 2k22/IT/121

NAME: ALISHBA MEMON

SEAT NUMBER 2k22/IT/18

Dr Muhammad Ali Memon

Supervisor's name and signature

Final Year Projects' Coordinator

Dr Muhammad Ali Nizamani

Co-Supervisor's name and signature

Final Year Projects' Committee Head

Chairman

Department of Information Technology

ACKNOWLEDGEMENTS

All praise and gratitude belong to Allah Almighty, whose countless blessings, guidance, and mercy enabled us to complete this project successfully. Without His will and support, this achievement would not have been possible.

We would like to express our deepest gratitude to our respected Supervisor, Mr. M. Ali Memon, for his invaluable guidance, continuous support, and insightful suggestions throughout the course of this project. His expertise, encouragement, and constructive feedback played a pivotal role in shaping the direction and quality of our work.

We are also sincerely thankful to our Co-Supervisor, Mr. M. Ali Nizamani, for his co-operation, technical advice, and assistance, which greatly contributed to the successful completion of this project.

Special appreciation is extended to Rohaan Ahmed, who provided significant help and guidance in the design and development of the hardware architecture. His practical insights and technical support were instrumental in building the hardware component of the MEDITWIN system.

We would also like to acknowledge our teachers, whose knowledge, motivation, and academic support laid the foundation for this project.

Finally, we are deeply grateful to our families for their unconditional support, patience, prayers, and encouragement throughout our academic journey. Their belief in us gave us the strength and motivation to overcome challenges and achieve our goals.

ABSTRACT

The increasing need for remote healthcare solutions has highlighted the importance of systems that support continuous patient monitoring. This project presents MEDITWIN, a digital twin-based healthcare monitoring system designed to observe and manage patient health conditions in real time. The system enables continuous tracking of vital parameters such as body temperature, pulse rate, and physical activity, providing timely insights into patient well-being.

MEDITWIN supports effective patient health management by maintaining structured health records and presenting them through an intuitive web-based dashboard. The platform allows users and healthcare personnel to view real-time data, analyze historical trends, and identify abnormal health patterns. An integrated alert mechanism assists in recognizing critical conditions and supports prompt medical attention when required.

By combining wearable sensor data with a virtual representation of patient health, MEDITWIN reduces the need for frequent physical clinical visits while enhancing continuous observation. Overall, the system aims to improve remote healthcare delivery, support early identification of health risks, and provide a reliable and user-friendly monitoring solution for modern healthcare environments.

LIST OF FIGURES

Figure 1-1: Context Diagram of MEDITWIN	3
Figure 3-1: Use Case Diagram of Patient	11
Figure 3-2: Use Case Diagram of admin	12
Figure 3-3: Use Case Diagram of sensor	13
Figure 3-4: Activity diagram of Patient	14
Figure 3-5: Activity diagram of Admin	15
Figure 3-6: Activity Diagram for Sensor	16
Figure 3-7: Entity Relationship Diagram	17
Figure 3-8: System Architecture	18
Figure 5-1: ESP32	28
Figure 5-2: MPU6050	29
Figure 5-3: MAX30102	30
Figure 5-4: DSB18B20	31
Figure 5-5: Prototyping	32
Figure 5-6: Log out confirmation	33
Figure 5-7: Historical Trends	34
Figure 5-8: Profile Interface	35
Figure 5-9: Patient Records	36
Figure 5-10: Dashboard	37

CONTENTS

CERTIFICATE	I
ACKNOWLEDGEMENTS	II
ABSTRACT	III
LIST OF FIGURES	IV
CONTENTS	5
Chapter No. 1 INTRODUCTION	1
1.1 BACKGROUND	1
1.2 WHAT IS TWIN TECHNOLOGY ?	1
1.3 OBJECTIVES	2
1.4 SCOPE	2
Chapter No. 2 Background and Literature Review	4
2.1 BACKGROUND	4
2.2 TWIN TECHNOLOGY	5
2.3 HARDWARE USED	5
2.4 MEDITWIN OVERVIEW	6
2.4.1 SYSTEM COMPONENT	6
2.5 HIGHLIGHTED FEATURES	6
2.5.1 ADMINISTRATOR FEATURES	7
2.5.2 PATIENT FEATURES	7
2.5.3 SENSOR USED AND THEIR FUNCTION	8
2.5.4 AREA OF APPLICATION	8
2.6 ADVANTAGES MEDITWIN	8
Chapter No. 3 ANALYSIS & DESIGN	9
3.1 REQUIREMENTS	9
3.1.1 FUNCTIONAL REQUIREMENTS	9
3.1.2 NON-FUNCTIONAL REQUIREMENTS	10
3.2 DESIGN OF MEDITWIN	11
3.2.1 USE CASE FOR PATIENTS	11
3.2.2 USE CASE FOR ADMIN	12
3.2.3 USE CASE FOR SENSORS	12
3.2.4 ACTIVITY DIAGRAM FOR PATIENT	14
3.2.5 ACTIVITY DIAGRAM FOR ADMIN	15
3.2.6 ACTIVITY DIAGRAM FOR SENSORS	16
3.3 ENTITY RELATIONSHIP DIAGRAM	17
3.4 SYSTEM ARCHITECTURE MEDITWIN	18

Chapter No. 4 TOOL & TECHNOLOGIES	19
4.1 OVERVIEW OF TOOLS	19
4.1.1 Hardware Components	19
4.1.2 Software Components	19
4.2 REACT.JS/CSS	20
4.2.1 CSS	20
4.3 NODE JS	21
4.3.1 USAGE NODE.JS	21
4.3.2 ADVANTAGES OF NODE.JS	21
4.4 EXPRESS.JS	22
4.4.1 ADVANTAGES OF EXPRESS.JS	22
4.5 MONGODB	23
4.7.2 MAX30102 SENSOR	26
4.7.3 DS18B20 TEMPERATURE SENSOR	26
4.7.4 MPU6050 SENSOR	26
4.7.5 OLED 4-INCH DISPLAY	26
4.7.6 ARDUINO PLATFORM	27
 Chapter No. 5 IMPLEMENTATION	 28
5.1 IMPLEMENTATION OF HARWARE	28
5.1.1 ESP32 MICRO-CONTROLLER	28
5.1.2 MPU6050	28
5.1.3 MAX30102	30
5.1.4 DS18B20	31
5.1.5 PROTOTYPING PLATFORM	33
5.1.6 LOGOUT CONFIRMATION	34
5.1.7 HISTORICAL TRENDS AND VISUALIZATION	35
5.1.8 PROFILE INTERFACE OF MEDITWIN	36
5.1.9 PATIENT RECORDS	37
5.1.10 DASHBOARD INTERFACE OF MEDITWIN	38
 Chapter No. 6 RESULTS AND CONCLUSION	 39
6.1 RESULT	39
6.2 CONCLUSION	39
6.3 FUTURE RECOMMENDATION	40
REFERENCES	41

1.1 BACKGROUND

The demand for continuous and remote healthcare monitoring has grown significantly in recent years due to increasing patient populations, chronic diseases, and limited healthcare resources. Traditional methods of health monitoring often require frequent hospital visits, which can be time-consuming, costly, and inconvenient, especially for patients with mobility challenges or those in remote areas. Advances in wearable technology, Internet of Things (IoT), and digital health platforms have enabled real-time monitoring of vital signs, providing timely data to healthcare providers and patients alike. These technologies support proactive healthcare management by allowing early detection of abnormal conditions and facilitating rapid medical intervention. Despite these advances, many systems lack integration between real-time data acquisition, historical record management, and patient-specific digital representations. The MEDITWIN project addresses this gap by creating a comprehensive digital twin-based system that continuously collects, analyzes, and visualizes patient health data, offering a reliable, user-friendly, and accessible solution for modern healthcare monitoring.

1.2 WHAT IS TWIN TECHNOLOGY?

Twin technology, commonly known as Digital Twin, is the creation of a virtual replica of a physical object, system, or process that mirrors its real-world counterpart in real time. In healthcare, this means building a virtual model of a patient that continuously reflects their physiological and health conditions based on real-time sensor data, historical records, and analytical models.

A digital twin collects data from wearable devices or IoT sensors, integrates it into a virtual representation, and enables monitoring, simulation, and predictive analysis. For example, changes in a patient's heart rate, body temperature, or activity levels can be reflected immediately in the digital twin, allowing healthcare providers to detect anomalies, predict potential risks, and make informed clinical decisions without constant physical examination.

In MEDITWIN, twin technology allows the creation of a real-time, interactive virtual model of the patient, which is central to remote monitoring, early risk detection, and efficient healthcare delivery.

1.3 AIM & OBJECTIVES

The primary objective of the MEDITWIN project is to develop a comprehensive digital twin-based healthcare monitoring system that enables continuous observation of patient health conditions. The system aims to provide real-time insights into vital parameters such as body temperature, pulse rate, and physical activity, allowing both patients and healthcare providers to make informed decisions.

Following are aims and objectives.

- To monitor patient health in real time.
- To create a digital twin that reflects patient physiological data.
- To provide a dashboard for visualizing health trends and records.
- To ensure secure storage of patient data and medical records.
- To support remote healthcare and reduce frequent clinical visits.

1.4 SCOPE

The MEDITWIN project focuses on providing a comprehensive platform for continuous and remote patient monitoring. The system allows real-time tracking of vital health parameters such as body temperature, pulse rate, and physical activity. It supports visualization of historical health data through a web-based dashboard and provides a digital twin for each patient, enabling accurate virtual representation and monitoring.

The system generates alerts for abnormal health conditions, ensuring timely intervention, and securely stores patient records and uploaded medical files for organized retrieval. MEDITWIN is designed to reduce dependency on frequent clinical visits by facilitating remote healthcare delivery. The platform is scalable, user-friendly, and suitable for both patients and healthcare providers, supporting proactive healthcare management and efficient decision-making.

The MEDITWIN project focuses on providing a comprehensive platform for continuous and remote patient monitoring. The system allows real-time tracking of vital health parameters such as body temperature, pulse rate, and physical activity. It supports visualization of historical health data through a web-based dashboard and provides a digital twin for each patient, enabling accurate virtual representation and monitoring.

The system generates alerts for abnormal health conditions, ensuring timely intervention, and securely stores patient records and uploaded medical files for organized retrieval. MEDITWIN is designed to reduce dependency on frequent clinical visits by facilitating remote healthcare delivery. The platform is scalable, user-friendly, and suitable for both patients and healthcare providers, supporting proactive healthcare management and efficient decision-making.

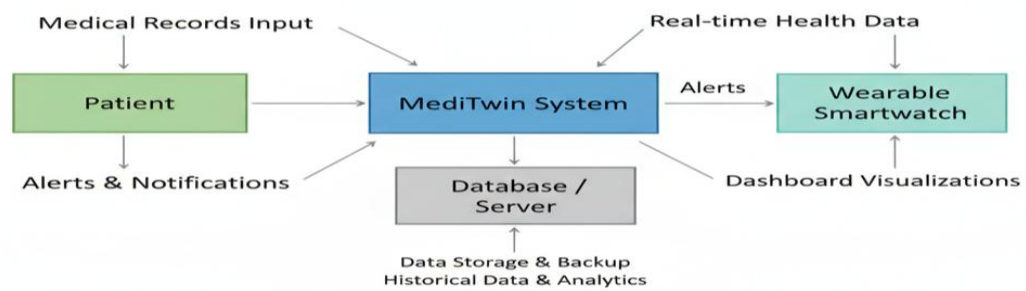


Figure 1-1: Context Diagram of MEDITWIN

Chapter No. 2

BACKGROUND AND LITERATURE REVIEW

2.1 BACKGROUND

The demand for efficient healthcare solutions has grown rapidly due to increasing populations, chronic illnesses, and the need for timely medical intervention. Traditional healthcare monitoring methods often require frequent hospital visits, which can be inconvenient, costly, and time-consuming, particularly for patients in remote areas or with limited mobility. Recent advancements in wearable technology, the Internet of Things (IoT), and digital health platforms have enabled continuous and real-time patient monitoring, allowing healthcare providers to track vital signs and respond promptly to health changes.

Despite these technological advances, many existing systems lack an integrated approach that combines real-time data acquisition, historical health records, and virtual patient representation. MEDITWIN addresses this gap by providing a digital twin-based healthcare monitoring system that collects, analyzes, and visualizes patient data efficiently. By enabling remote monitoring and proactive healthcare management, MEDITWIN aims to improve patient outcomes, reduce the need for frequent clinical visits, and support healthcare providers in delivering timely, data-driven interventions.

2.2 TWIN TECHNOLOGY

Twin technology [1], commonly referred to as digital twin, is the creation of a virtual representation of a physical entity or system that reflects its real-time status, behavior, and performance. In healthcare, a digital twin acts as a virtual model of a patient, continuously receiving data from wearable devices or IoT sensors to mirror physiological conditions such as heart rate, body temperature, and physical activity.

By maintaining an accurate virtual representation, digital twins enable continuous monitoring, predictive analysis, and simulation without requiring constant physical examination. Healthcare providers can track health trends, detect abnormalities early, and make informed decisions based on real-time and historical data.

In the context of MEDITWIN, twin technology allows the system to create a dynamic virtual model of the patient, integrating real-time sensor data, historical health records, and alerts. This approach enhances remote healthcare monitoring, supports early risk detection, and improves the overall efficiency of patient management.

2.3 HARDWARE USED

- Temperature Sensor: **DBS18B20** Measures body temperature accurately for detecting fever or other health anomalies.
- Pulse/Heart Rate Sensor: **MAX30102** Monitors heart rate to track cardiovascular health and detect irregularities.
- Accelerometer / Step Counter: **SPU6050** Tracks physical activity, steps, and movement patterns.
- Microcontroller Unit (MCU): **ESP32** Processes sensor data and communicates with the backend system.

- Connectivity Module: **ARDIUNO** Enables data transmission from the wearable to the MEDITWIN platform for real-time visualization and storage.

2.4 MEDITWIN OVERVIEW

MEDITWIN is a digital twin-based healthcare monitoring system designed to provide continuous patient monitoring, real-time health insights, and efficient healthcare management. The system integrates wearable devices, a web-based dashboard, and a secure backend to ensure accurate tracking, visualization, and storage of patient data.



Figure 5-10: Dashboard

2.4.1 SYSTEM COMPONENT

- Web-Based Dashboard: Displays health trends, alerts, and patient records for both patients and administrators.
- Backend Infrastructure: Handles secure data storage, alert generation, and API communication between frontend and devices.

- Digital Twin Model: Maintains a virtual representation of each patient for continuous monitoring and predictive analysis

2.5 HIGHLIGHTED FEATURES

1. Administrators
2. Patient
3. Sensors for IOT device

2.5.1 ADMINISTRATOR FEATURES

- Patient Data Management: Add, edit, or review patient profiles and health records.
- Alert Monitoring: Track critical alerts and intervene when necessary.
- Dashboard Overview: Comprehensive view of all patients' data and health trends.
- Data Security Management: Ensure secure access to patient records and uploaded files.
- System Scalability: Manage multiple patients simultaneously with efficient system performance.
- Patient Data Management: Add, edit, or review patient profiles and health records.
- Alert Monitoring: Track critical alerts and intervene when necessary.
- Dashboard Overview: Comprehensive view of all patients' data and health trends.
- Data Security Management: Ensure secure access to patient records and uploaded files.
- System Scalability: Manage multiple patients simultaneously with efficient system performance.

2.5.2 PATIENT FEATURES

- **Real-Time Health Data Access:** Patients can view their body temperature, pulse rate, and physical activity continuously.
- **Historical Health Records:** Access to past health trends and recorded vitals for self-monitoring.

- **Alert Notifications:** Receive automatic alerts if any health parameter crosses predefined thresholds.
- **User-Friendly Dashboard:** Simple interface to track and understand health data easily.
- **Secure File Upload:** Upload medical reports or documents to the system for record-keeping.

2.5.3 SENSOR USED AND THEIR FUNCTION

- **Temperature Sensor (LM35 / DS18B20):** Measures body temperature to detect fever or abnormal conditions.
- **Pulse/Heart Rate Sensor (MAX30100 / MAX30102):** Monitors heart rate and blood oxygen levels to track cardiovascular health.
- **Accelerometer (ADXL345 / MPU6050):** Measures body movement, orientation, and physical activity levels.
- **Microcontroller Unit (Arduino / ESP32 / STM32):** Processes all sensor data and communicates with the backend server.

2.5.4 AREA OF APPLICATION

MEDITWIN can be applied in remote patient monitoring, chronic disease management, post-surgery care, and elderly healthcare, allowing continuous tracking of vital health parameters. It is also useful for fitness and wellness tracking, preventive healthcare, and clinical research, providing timely insights and supporting early intervention for improved patient outcomes.

2.6 ADVANTAGES MEDITWIN

- Saving of time & man power
- Enables real-time health monitoring.
- Reduces the need for frequent hospital visits.
- Supports early detection of health issues.

3.1 REQUIRMENTS

A requirement is an objective that must be met. Planners cast most requirements in functional terms, leaving design and implementation details to the developers. They may specify price, performance, and reliability objectives in fine detail, along with some aspects of the user interface. Sometimes, they describe their objectives more precisely than realistically. There are two types of requirements given below:

3.1.1 FUNCTIONAL REQUIREMENTS

A requirement that specifies a function that a system or component must be able to perform is called functional requirement. These include inputs, outputs, calculations, external interfaces, communications, and special management information needs. Functional requirements are also called behavioral requirements because they address what the system does.

In many cases, if the user requirements are written for the requestor and not the end-user, the functional requirements are combined with the functional requirements; this is common within companies that have a strong Information Technology department that is tasked with doing the work. The functional requirements of MEDITWIN include:

- The system shall collect real-time data from health sensors.
- The system shall display patient vitals on the dashboard.
- The system shall store patient health records securely.
- The system shall allow patient registration and login.
- The system shall track historical health data trends

3.1.2 NON-FUNCTIONAL REQUIREMENTS

Non-Functional Requirements in Software Engineering presents a systematic and pragmatic approach to building quality into software systems. Systems must exhibit software quality attributes, such as accuracy, performance, security and modifiability. However, such non-functional requirements are difficult to address in many projects, even though there are many techniques to meet functional requirements in order to provide desired functionality.

Non-Functional Requirements in Software Engineering is an excellent resource for software engineering practitioners, researchers and students. For example, software performance requirements, software external interface requirements, software design constraints, and software quality attributes. Nonfunctional requirements are difficult to test; therefore, they are usually evaluated subjectively. ^[5]

The Dynamic Interface Of MEDITWIN includes:

- The system shall ensure high data security and privacy.
- The system shall provide fast system response time.
- The system shall support reliable real-time data updates.
- The system shall be scalable for future feature expansion.
- The system shall be easy to use and user-friendly.

3.2 DESIGN OF MEDITWIN

3.2.1 USE CASE FOR PATIENTS

The Patient Use Case Diagram shows how a patient interacts with MEDITWIN. The patient can register/login, view the dashboard, monitor vital signs like temperature, heart rate, and steps, and track health trends. They also receive alerts for abnormal readings, upload medical records, and access reports. This diagram highlights the patient's role in continuous monitoring and proactive health management, reducing the need for frequent hospital visits

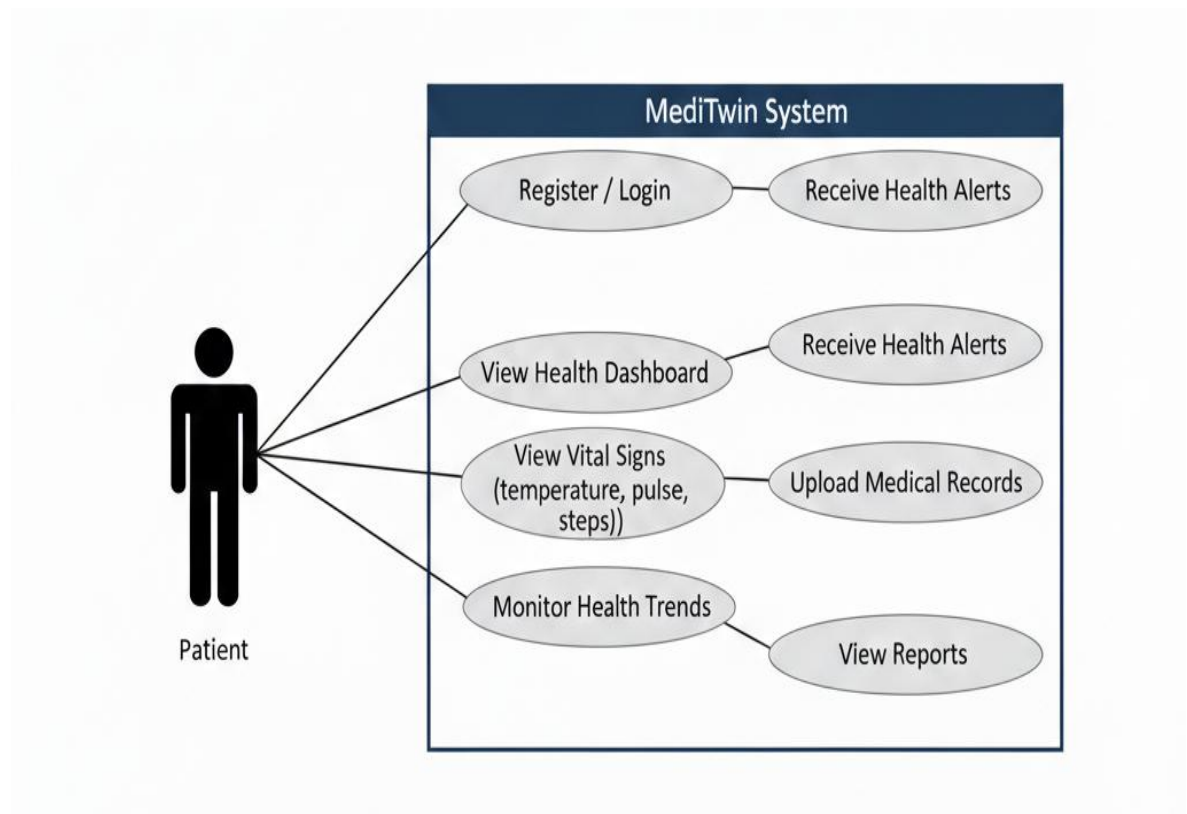


Figure 3-1: Use Case Diagram of Patient

3.2.2 USE CASE FOR ADMIN

The Admin Use Case Diagram shows how administrators interact with the MEDITWIN system. Admins can log in, manage patient accounts, monitor real-time sensor data, review alerts and medical records, and generate reports. They also manage system data and ensure smooth operation. This diagram highlights the admin's role in overseeing the system, maintaining data integrity, and supporting efficient healthcare management.

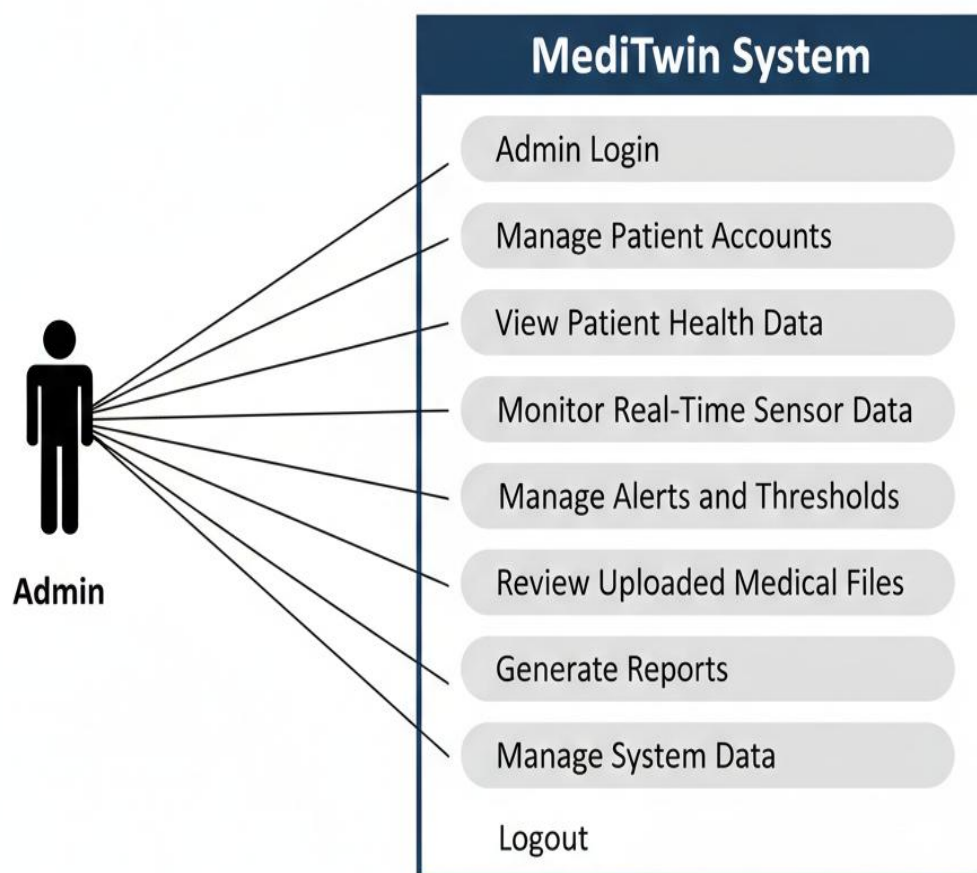


Figure 3-2: Use Case Diagram of Admin

3.2.3 USE CASE FOR SENSORS

The Sensors Use Case Diagram illustrates how the heart rate sensor, temperature sensor, step counter, and ESP32 microcontroller interact with the MEDITWIN system. Sensors measure vital signs and collect data, while the ESP32 processes and transmits the readings to the backend. The system then stores the data, updates the patient's digital twin, and triggers alerts if necessary. This diagram highlights the role of sensors in continuous health monitoring and accurate real-time data collection.

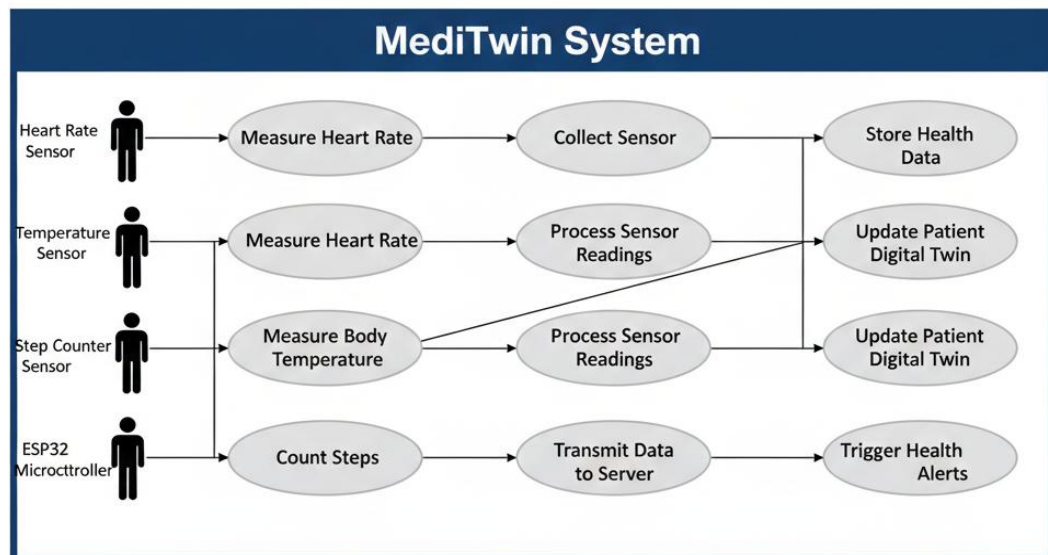


Figure 3-3: Use Case Diagram of Sensors

3.2.4 ACTIVITY DIAGRAM FOR PATIENT

To perform the activities patient has to login first after login he / she can check the vitals on the webpage, as shown in the following figure:

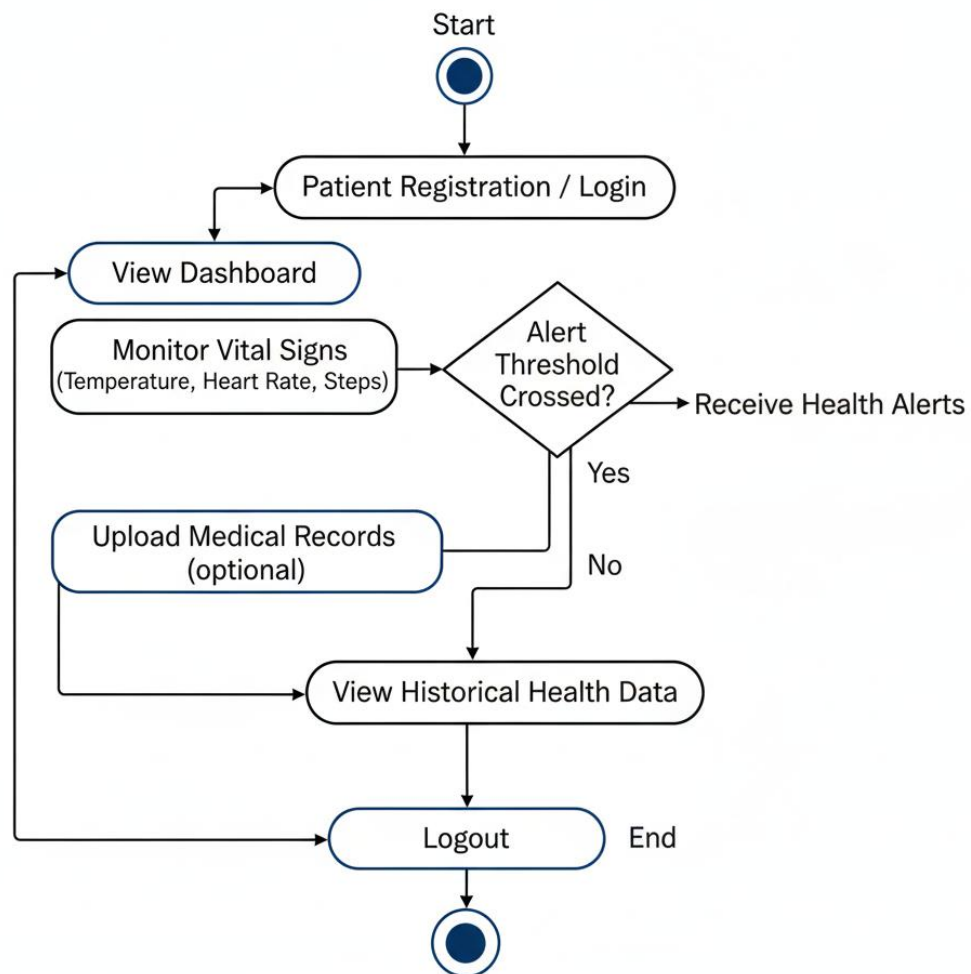


Figure 3-4: Activity diagram of Patient

3.2.5 ACTIVITY DIAGRAM FOR ADMIN

The Admin Activity Diagram shows how administrators log in, manage patient accounts, monitor sensor data, review alerts and records, generate reports, and log out. It highlights the admin's role in overseeing the system and ensuring efficient healthcare management

MediTwin

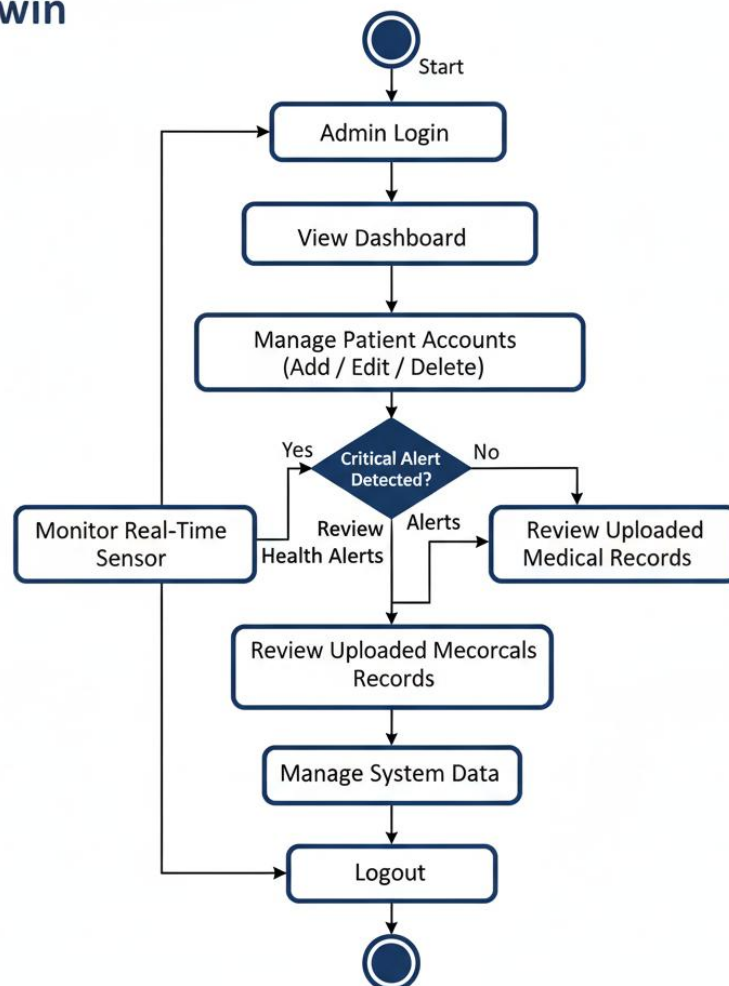


Figure 3-5: Activity diagram of Admin

3.2.6 ACTIVITY DIAGRAM FOR SENSORS

The Sensors Activity Diagram shows how heart rate, temperature, and step sensors measure data, which is collected and processed by the ESP32, transmitted to the backend, stored, and used to update the patient's digital twin. It highlights the sensors' role in continuous health monitoring and real-time alert generation.

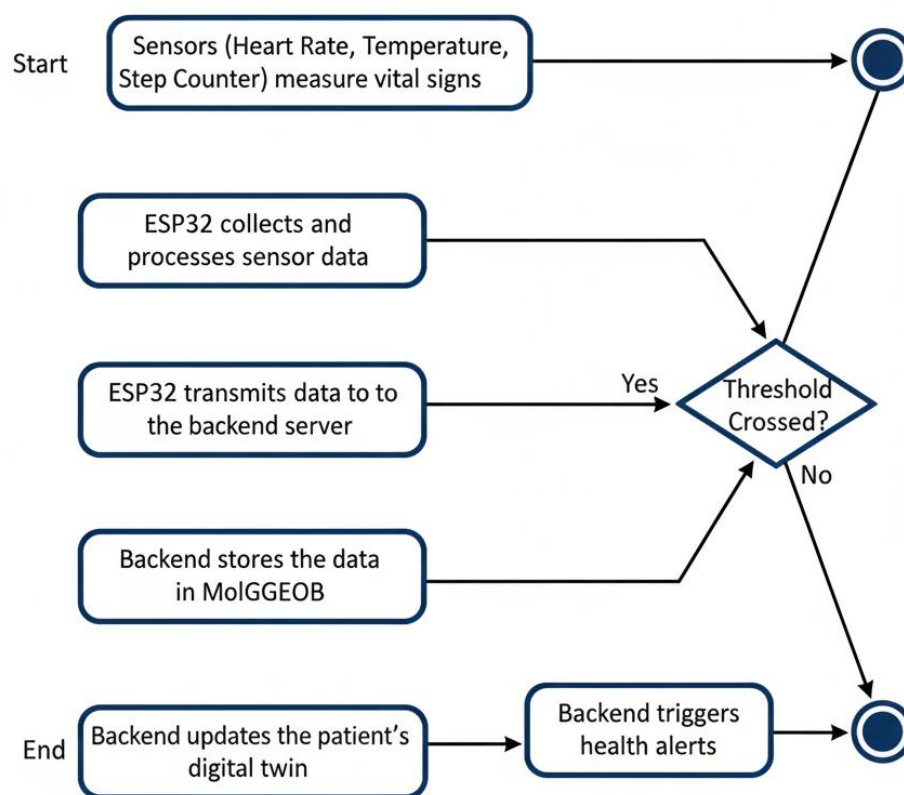


Figure 3-6: Activity Diagram for Sensors

3.3 ENTITY RELATIONSHIP DIAGRAM

An entity-relationship diagram (ERD) is a graphical representation of an information system that shows the relationship between people, objects, places, concepts or events within that system. An ERD is a data modeling technique that can help define business processes and can be used as the foundation for a relational database. Three main components of an ERD are the entities, which are objects or concepts that can have data stored about them, the relationship between those entities, and the cardinality, which defines that relationship in terms of numbers.

ER Diagram

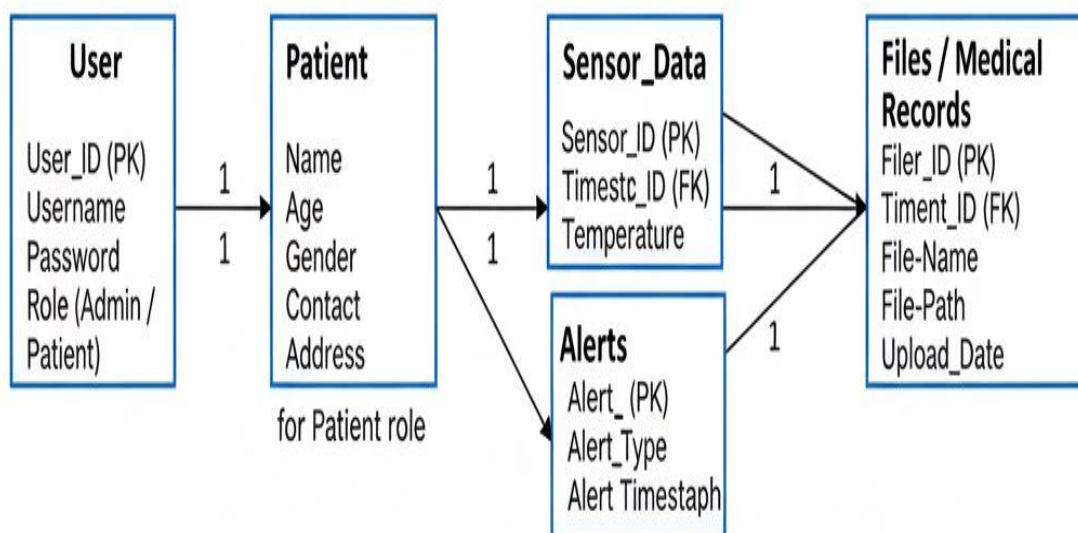


Figure 3-7: Entity Relationship Diagram

3.4 SYSTEM ARCHITECTURE MEDITWIN

The system architecture of MEDITWIN explains how hardware layer is transmitting data to the backend layer. With the help of API calls data is being fetched which is later displaying on front end MEDITWIN

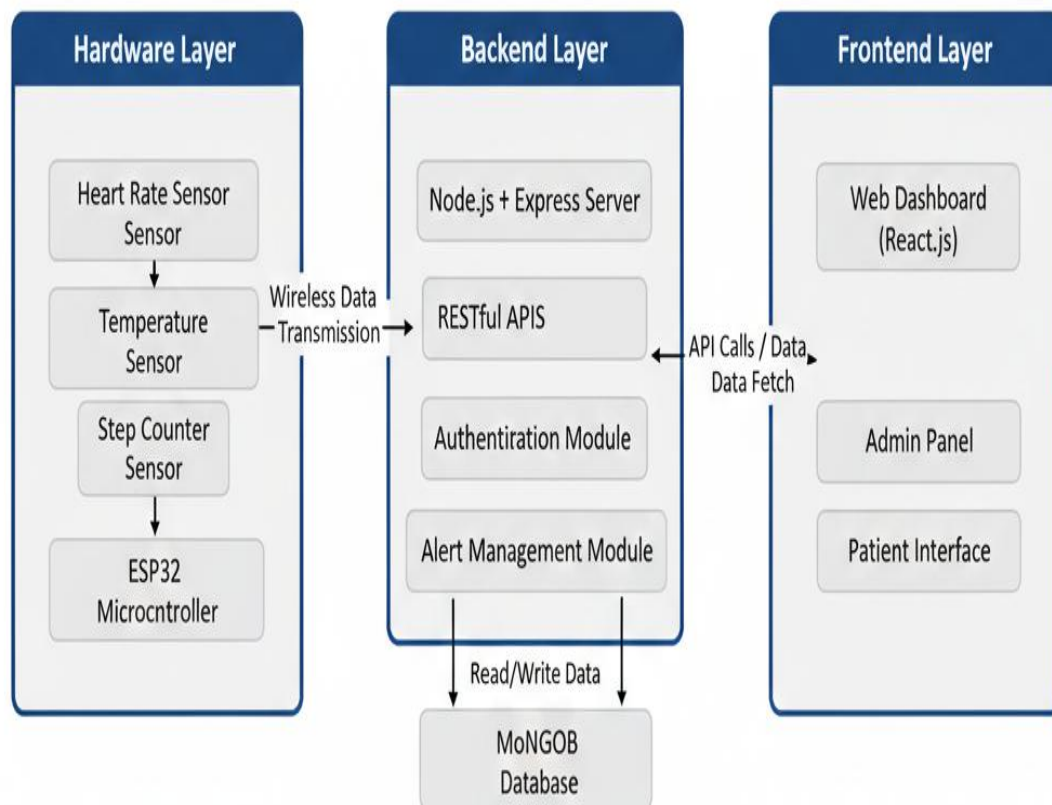


Figure 3-8: System Architecture

Chapter No. 4

TOOL & TECHNOLOGIES

4.1 OVERVIEW OF TOOLS

The MEDITWIN system combines **hardware sensors, microcontrollers, and software technologies** to enable real-time health monitoring and management.

4.1.1 Hardware Components

- **ESP32 Microcontroller:** Acts as the central processing unit for sensor data, collecting, processing, and transmitting readings to the backend server.
- **MAX30102 Sensor:** Measures **heart rate and oxygen saturation** accurately for continuous patient monitoring.
- **DS18B20 Temperature Sensor:** Monitors **body temperature** in real-time.
- **MPU6050 Sensor:** Captures **motion and orientation**, tracking physical activity such as steps and movement patterns.
- **OLED 4-inch Display:** Provides a **local interface** for real-time visualization of sensor data.
- **Arduino Platform:** Used for **prototyping and testing sensor integration** on a breadboard before final hardware implementation.

4.1.2 Software Components

- **Frontend:** Developed using React.js, providing an interactive and responsive dashboard for patients and administrators. Features include live sensor data visualization,
- **Backend:** Built with Node.js and Express.js, offering RESTful APIs for data management, user authentication, alert generation, and file handling.
- **Database:** MongoDB, integrated via Mongoose, stores patient data, sensor readings, alerts, and file metadata securely.

- Additional Tools: Multer for file uploads, dotenv for secure environment variables, cors for frontend-backend communication, Nodemon for development, and Postman for API testing.

This combination of hardware sensors and software technologies allows MEDITWIN to provide real-time health monitoring, secure data management, alert notifications, and an intuitive user interface, making it a comprehensive healthcare monitoring solution

4.2 REACT.JS/CSS

React.js is a modern JavaScript library developed by Facebook for building dynamic and responsive user interfaces. It follows a component-based architecture, allowing developers to create reusable UI components such as buttons, forms, dashboards, and charts. This modular structure improves code maintainability, scalability, and reusability. React uses a virtual DOM, which efficiently updates only the parts of the webpage that change, resulting in faster performance and smoother user interactions. It also supports state management, which helps track and manage data within components, making applications interactive and responsive. React is widely used for developing web applications, single-page applications (SPAs), and dashboards in both commercial and research projects.

4.2.1 CSS

Cascading Style Sheets (CSS) is a stylesheet language used to design and style web pages. It allows developers to control the layout, colors, fonts, spacing, and overall appearance of HTML elements.. By separating content (HTML) from design (CSS), developers can maintain a clean code structure and ensure consistent styling across multiple pages. CSS is essential for designing intuitive dashboards, professional layouts, and visually appealing interfaces, which are key for applications like MEDITWIN.

4.3 NODE JS

Node.js is an open-source, JavaScript runtime environment that allows developers to execute JavaScript code on the server side. It is built on the V8 JavaScript engine and is designed for building scalable, fast, and efficient network applications. Node.js uses an event-driven, non-blocking I/O model, which makes it lightweight and ideal for handling multiple simultaneous connections without slowing down the server [1].

4.3.1 USAGE NODE.JS

- Fast and efficient with non-blocking I/O
- Scalable for multiple connections
- Same language for frontend and backend
- Large library ecosystem (npm)
- Supports real-time applications

4.3.2 ADVANTAGES OF NODE.JS

- Fast and efficient with non-blocking I/O
- Scalable for multiple connections
- Same language for frontend and backend
- Large library ecosystem (npm)
- Supports real-time applications

4.4 EXPRESS.JS

Express.js is a lightweight, open-source web framework for Node.js that simplifies the process of building server-side applications and APIs. It provides a robust set of features for routing, middleware integration, and handling HTTP requests and responses, allowing developers to create modular and scalable backend systems. Express.js is widely used for building RESTful APIs, web servers, and backend services that connect to databases and frontend applications. Its minimalist design ensures flexibility, while middleware support allows additional functionality such as authentication, logging, and file handling to be easily integrated. In projects like MEDITWIN, Express.js enables seamless communication between the frontend dashboard and the backend database, ensuring efficient and secure management of patient data and real-time health monitoring [2].

4.4.1 ADVANTAGES OF EXPRESS.JS

- Lightweight and minimal framework
- Simplifies routing and request handling
- Middleware support for modular code
- Easy integration with databases
- Fast and efficient for server-side development

4.4.2 USES OF EXPRESS.JS

- Building RESTful APIs
- Backend for web applications
- Handling HTTP requests and responses
- Connecting frontend to database
- Managing authentication and file uploads

4.5 MONGODB

MongoDB is a NoSQL, document-oriented database that stores data in JSON-like documents instead of traditional tables. It is designed for high performance, scalability, and flexibility, making it ideal for handling large volumes of unstructured or semi-structured data. MongoDB allows developers to easily manage complex datasets, perform queries, and integrate with modern applications, such as real-time healthcare monitoring systems like MEDITWIN [3].

4.5.1 ADVANTAGES

- Flexible schema design for dynamic data
- High performance and fast queries
- Easily scalable for large datasets
- Supports complex queries and indexing
- Open-source with large community support

4.5.2 USES

- Storing patient health records and sensor data
- Managing real-time data in web applications
- Backend for RESTful APIs
- Storing metadata for files and alerts
- Integration with Node.js/Express backend

4.6 OVERVIEW OF OTHER BACKEND TOOLS

4.6.1 MONGOOSE

Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js. It

provides a schema-based solution to model application data, enforce validation, and simplify database operations [4].

- **Advantages:** Easy schema definition, data validation, query simplification, middleware support, integrates seamlessly with Node.js.
- **Uses:** Defining patient data and alert schemas, querying MongoDB, handling relationships between collections, validating data before saving.

4.6.2 DOTENV

dotenv is a Node.js module that allows developers to manage environment variables securely. It helps store sensitive information such as database URIs, API keys, and server ports outside the codebase [5].

- **Advantages:** Secure management of configuration, prevents hardcoding secrets, easy to configure for multiple environments.
- **Uses:** Storing MongoDB connection strings, server ports, and API keys for MEDITWIN backend.

4.6.3 CORS

CORS (Cross-Origin Resource Sharing) is a middleware used to allow secure communication between frontend and backend hosted on different domains [6].

- **Advantages:** Enables cross-origin requests, simple configuration, improves security.
- **Uses:** Allowing React.js frontend to access Node.js/Express APIs.

4.6.4 MUTLER

Multer is a Node.js middleware for handling multipart/form-data, primarily used for file uploads [7].

- **Advantages:** Efficient file handling, easy storage configuration, supports metadata saving.

- **Uses:** Uploading medical records and storing file references in MongoDB for MEDITWIN.

4.6.5 NODEMON

Nodemon is a development tool that automatically restarts the Node.js server when file changes are detected [8].

- **Advantages:** Speeds up development, reduces manual restarts, improves productivity.
- **Uses:** Backend development and testing of MEDITWIN APIs.

4.6.6 POSTMAN

Postman is a tool used to test and debug APIs. It allows sending HTTP requests to the backend and inspecting responses [9].

- **Advantages:** Easy API testing, supports automated tests, clear visualization of requests/responses.
- **Uses:** Testing MEDITWIN RESTful APIs for authentication, patient data, alerts, and file uploads.

4.7 OVERVIEW OF HARDWARE TOOLS

The hardware component of MEDITWIN is designed to collect, process, and transmit real-time patient health data. The system combines sensors, microcontrollers, and display modules, providing accurate monitoring and a platform for future wearable integration.

4.7.1 ESP32 MICROCONTROLLER

The ESP32 is a low-cost, high-performance microcontroller with Wi-Fi and Bluetooth capabilities, making it ideal for IoT-based health monitoring. It acts as the central processing unit, collecting data from all connected sensors, processing it, and transmitting it to the backend server. Its dual-core processor and multiple GPIO pins allow simultaneous sensor readings and local processing [12].

4.7.2 MAX30102 SENSOR

The MAX30102 is an optical sensor module used for measuring heart rate and blood oxygen saturation (SpO₂). It uses LEDs and photodetectors to detect variations in blood flow, providing accurate readings for continuous health monitoring [13].

- **Advantages:** High accuracy, small form factor, low power usage.
- **Uses:** Monitoring real-time heart rate and SpO₂ levels for patients.

4.7.3 DS18B20 TEMPERATURE SENSOR

The DS18B20 is a digital temperature sensor used to measure body temperature accurately. It communicates via a 1-Wire interface, making it easy to connect to microcontrollers [14].

- **Advantages:** Precise readings, simple digital output, integration with ESP32.
- **Uses:** Tracking real-time body temperature for fever or abnormal conditions.

4.7.4 MPU6050 SENSOR

The MPU6050 is a 6-axis accelerometer and gyroscope module. It measures motion, orientation, and steps, allowing physical activity monitoring [15].

- **Advantages:** Compact size, accurate motion detection.
- **Uses:** Monitoring patient movement, step counts.

4.7.5 OLED 4-INCH DISPLAY

A 4-inch OLED display is used to provide a local interface for patients or caregivers. It visually presents real-time sensor data such as temperature, heart rate, and steps directly on the device [16].

- **Advantages:** High contrast, low power consumption, clear visualization.
- **Uses:** Displaying live readings from sensors for immediate observation.

4.7.6 ARDUINO PLATFORM

The Arduino platform is used for prototyping and testing sensor integration on a breadboard before moving to the final wearable device. It provides a simple interface for programming microcontrollers and verifying sensor accuracy [17].

- **Advantages:** Easy to program, vast community support, flexible prototyping.
- **Uses:** Testing and validating sensor data and connectivity with ESP32.

These hardware components collectively form the core of the MEDITWIN system, enabling accurate, real-time health monitoring. Currently, the sensors are mounted on a breadboard for development and testing, with plans to integrate them into a wearable smartwatch form factor in future iterations.

5.1 IMPLEMENTATION OF HARWARE

5.1.1 ESP32 MICRO-CONTROLLER

Below is the ESP 32 image which is used in a project. ESP32 microcontroller low cost, high performance IOT based health monitoring with Wi-fi and Bluetooth capabilities. It collects data from all connected sensors, processing it and transmitting it to the backend server.



Figure 5-1: ESP32

5.1.2 MPU6050

Below is the image of MPU6050 for motion and step detection. 6 axis accelerometer and gyroscope module. It measures motion, orientation and steps allowing physical activity monitoring.



Figure 5-2: MPU6050

5.1.3 MAX30102

Heart rate and SpO₂ monitoring. It uses LEDs and photodetectors to detect variations in blood flow, providing accurate reading for continuous health monitoring.



Figure 5-3: MAX30102

5.1.4 DS18B20

Temperature measurement sensor for assessing body temperature. It communicates via 1-wire interface, making it easy to connect to microcontrollers. It is prominent for tracking fever and abnormal condition



Figure 5-4: DSB18B20

5.1.5 PROTOTYPING PLATFORM

Illustrated all components connected for testing [18]. Output of vitals are shown clearly on the image such as oxygen level and hear-rate by utilizing sensors properly.

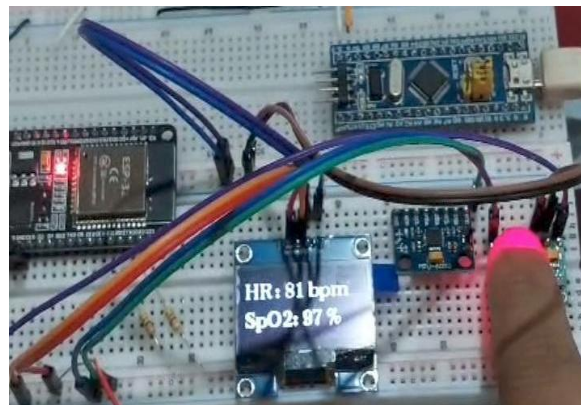


Figure 5-5: PROTOTYPING

5.1.5 LOGOUT CONFIRMATION

This figure illustrates the logout confirmation dialog of the MEDITWIN system. It ensures secure session handling by prompting the user to confirm before exiting the system, reducing accidental logouts and improving overall user experience.

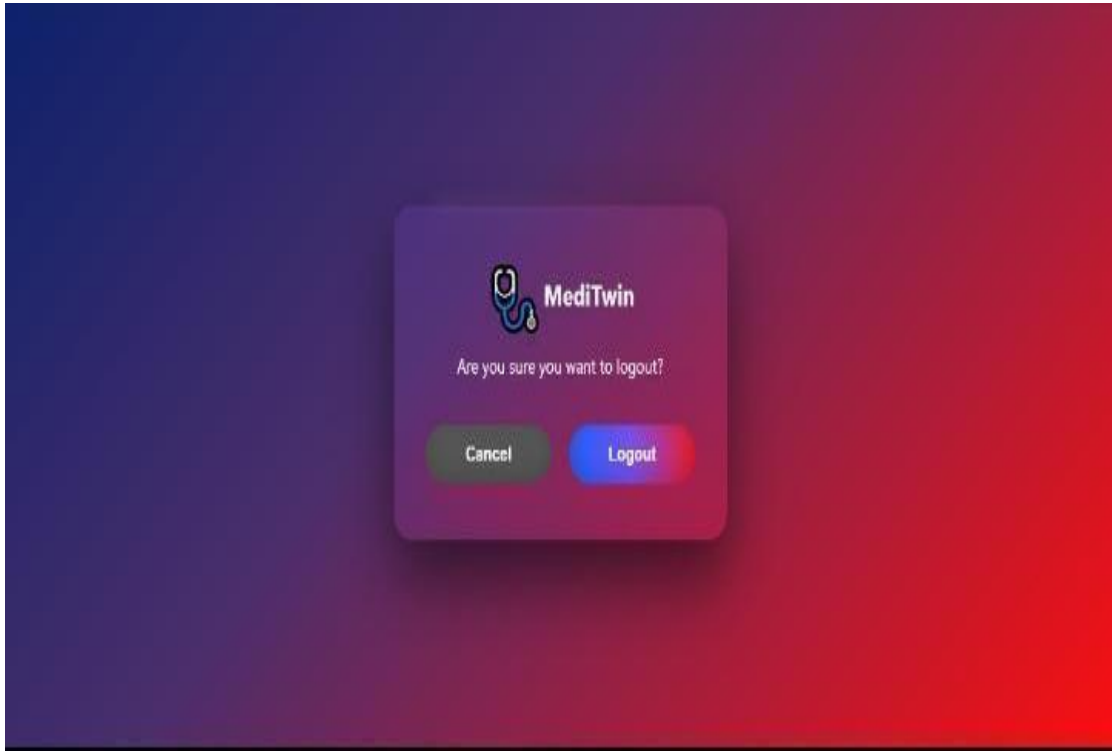


Figure 5-6: Logout Confirmation

5.1.6 HISTORICAL TRENDS AND VISUALIZATION

This figure presents the historical trend analysis of patient vital signs, including body temperature and heart rate. The graphical representation helps users easily observe variations and patterns over time, supporting better health monitoring and informed decision-making.

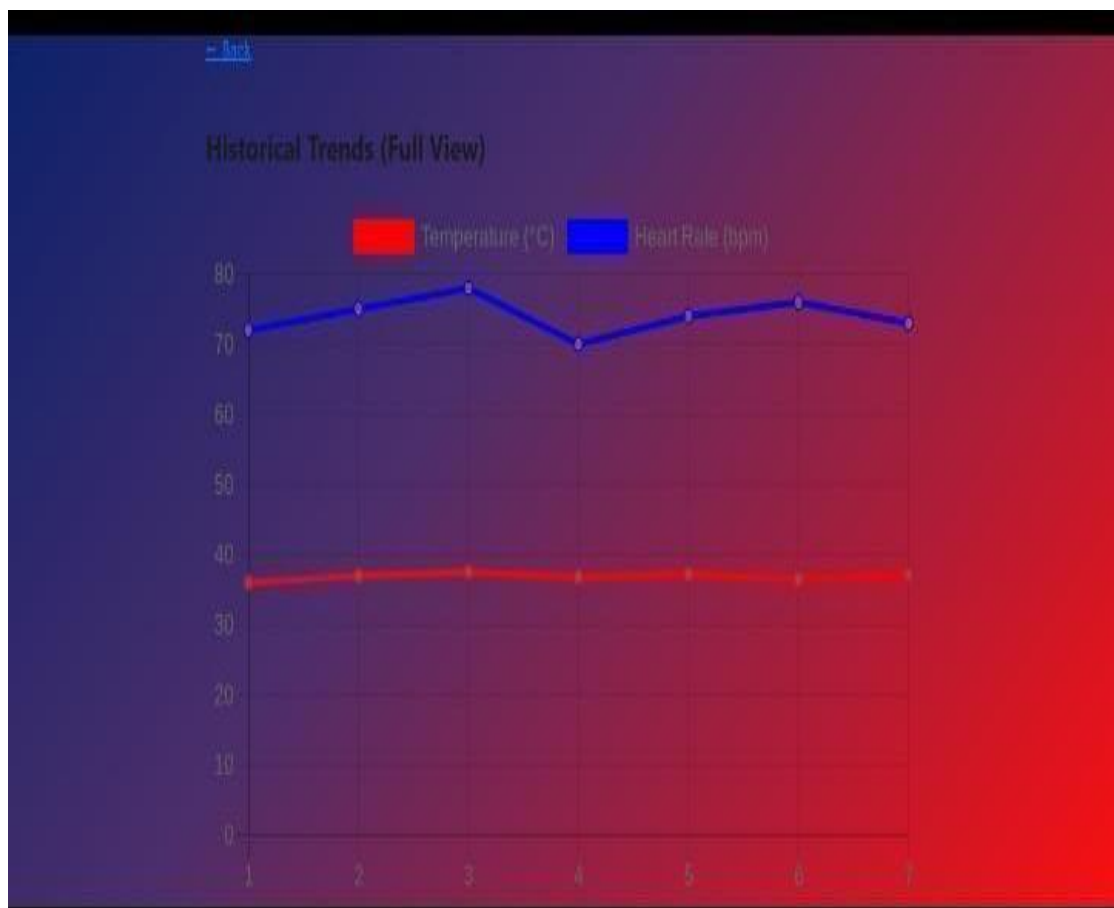


Figure 5-7: Historical Trends

5.1.7 PROFILE INTERFACE OF MEDITWIN

This figure shows the patient profile screen displaying essential patient information such as name, age, gender, patient ID, and latest vital readings. It also provides access to medical records, enabling centralized and organized patient data management within the system.

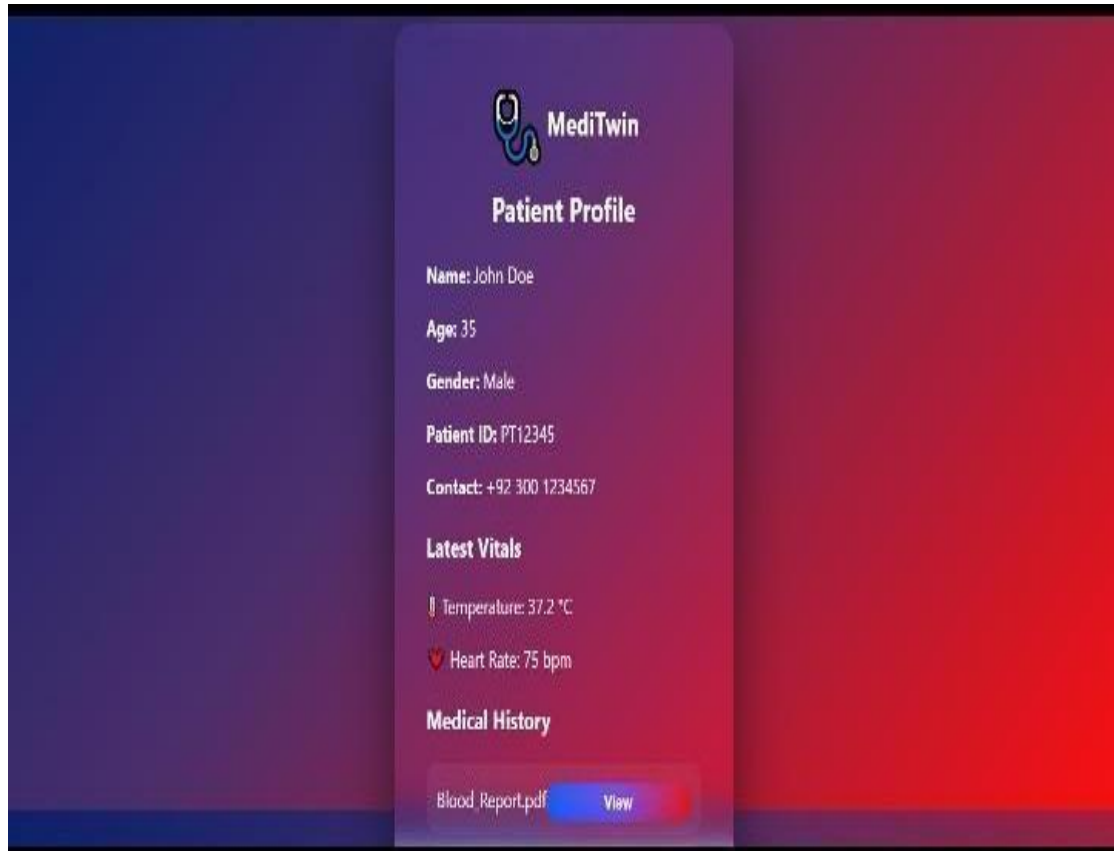


Figure 5-8: Profile Interface

5.1.8 PATIENT RECORDS

This figure show patients record in MEDITWIN from where patient can download his previous data which is monitored by sensors. It helps them to track and analyze the pattern of their overall health.



Figure 5-9: Patient Records

5.1.9 DASHBOARD INTERFACE OF MEDITWIN

This figure shows a twin technology and highlighting vitals [19] [20]. Digital replica of the patient is marking the crucial vitals such as temperature and heart rate of the patient. Below histogram is behaving according to real-time data of the user.



Figure 5-10: Dashboard

Chapter No. 6

RESULTS AND CONCLUSION

6.1 RESULT

The results obtained from the MEDITWIN system demonstrate its effectiveness in real-time health monitoring, data management, and alert generation. The system successfully collects data from wearable sensors, processes it via the ESP32 microcontroller, and displays it accurately on the web-based dashboard. Patients' vital signs, including temperature, heart rate, and physical activity, were monitored continuously, and alerts were triggered promptly whenever thresholds were crossed. The system also allowed secure storage and retrieval of historical health records and uploaded medical files. Overall, the results highlight MEDITWIN's ability to provide reliable, accurate, and timely health information, supporting both patients and administrators in proactive healthcare management and reducing dependency on frequent hospital visits

6.2 CONCLUSION

The MEDITWIN system demonstrates the potential of digital twin technology in healthcare by enabling continuous monitoring, real-time data visualization, and timely alerts. It provides patients with easy access to their health information while allowing administrators to efficiently manage and analyze patient data. The integration of wearable sensors, the ESP32 microcontroller, and a secure backend ensures accurate data collection, storage, and processing. Overall, MEDITWIN supports proactive healthcare management, early detection of health issues, and improved patient outcomes, highlighting its significance as a scalable and reliable healthcare monitoring solution for both personal and clinical use.

6.3 FUTURE RECOMMENDATION

Based on the current implementation of MEDITWIN, several future improvements and recommendations can be considered to enhance functionality, usability, and scalability:

1. **Wearable Integration:** Currently, sensors are mounted on a breadboard for testing and data collection. Future work should focus on integrating all sensors into a compact, ergonomic smartwatch form factor, improving portability, comfort, and continuous monitoring capability.[23]
2. **Sensor Optimization:** Explore the use of more advanced or low-power sensors to improve accuracy, reduce energy consumption, and extend battery life for long-term monitoring.
3. **Advanced Data Analytics:** Implement predictive analytics and machine learning algorithms on collected sensor data to identify trends, predict health risks, and provide personalized health recommendations.
4. **Mobile Application:** Develop a mobile app to complement the web dashboard, enabling patients to access real-time data, alerts, and historical trends on-the-go.
5. **Cloud Integration:** Migrate backend and database services to a cloud platform for better scalability, security, and remote access, allowing the system to handle a larger number of patients simultaneously.
6. **Enhanced Alert System:** Improve the alert module to include multi-channel notifications (SMS, email, push notifications) and priority-based alerts for critical health conditions [22].
7. **Battery and Power Management:** Focus on optimizing power consumption of the wearable device, potentially integrating rechargeable or energy-harvesting components to ensure long-term use.
8. **User Feedback and Usability Testing:** Conduct usability studies with real users to refine the interface, dashboard, and smartwatch design, ensuring the system is intuitive and easy to operate for patients of all ages.
9. **Integration with Healthcare Systems:** Enable compatibility with hospital EHR systems[24] and telemedicine platforms, allowing healthcare providers to access patient data seamlessly for clinical decision-making.

10. Additional Health Parameters: Expand sensor capabilities to monitor blood oxygen levels, ECG in detail, or stress indicators, making the system more comprehensive in health monitoring.

By implementing these recommendations, MEDITWIN can evolve from a prototype-based monitoring system into a fully functional, wearable, and scalable healthcare solution, offering both patients and healthcare providers enhanced reliability, convenience, and proactive care capabilities.

REFERENCES

- [1] Node.js Foundation, *Node.js Official Documentation*, 2024.
- [2] Express.js, *Express.js Official Website*, 2019.
- [3] MongoDB Inc., *MongoDB Node.js Driver Documentation*, 2017.
- [4] Automattic Inc., *Mongoose (MongoDB ODM)*, 2022.
- [5] dotenv, *dotenv (Node.js Environment Variables)*, 2024.
- [6] OpenJS Foundation, *cors (Node.js CORS Middleware)*, 2024.
- [7] Express Middleware, *Multer (Node.js File Upload Middleware)*, 2024.
- [8] Remy Sharp, *Nodemon (Node.js Utility)*, 2018.
- [9] Postman Inc., *Postman (API Development Platform)*, 2018.
- [10] Meta Platforms Inc., *React.js Official Documentation*, 2018.
- [11] World Wide Web Consortium (W3C), *CSS (Cascading Style Sheets) Reference*, 2025.

- [12] Espressif Systems, *ESP32 Series Datasheet*, 2022.
- [13] Analog Devices, *MAX30102 Sensor Datasheet*, 2022.
- [14] Maxim Integrated, *DS18B20 Temperature Sensor Datasheet*, 2023.
- [15] InvenSense/TDK, *MPU-6050 IMU Datasheet*, 2023.
- [16] Solomon Systech, *SSD1306 OLED Display Driver*, 2020.
- [17] Arduino, *Arduino Open-Source Hardware Platform*, 2020.
- [18] M. Kabir *et al.*, “Digital Twin in Healthcare IoT,” 2025.
- [19] S. Parween *et al.*, “Digital Twin for Movement Monitoring,” 2025.
- [20] Nasir *et al.*, “IoT-Based Healthcare Monitoring,” 2022.
- [21] Aladdin *et al.*, “Wearable IoT in Health,” 2026.
- [22] Baiense *et al.*, “Healthcare Monitoring with IoT & Wearables,” 2025.
- [23] Lu *et al.*, “Wearable Health Devices Review,” 2020.
- [24] Tan *et al.*, “Remote Patient Monitoring,” 2024.

THE END